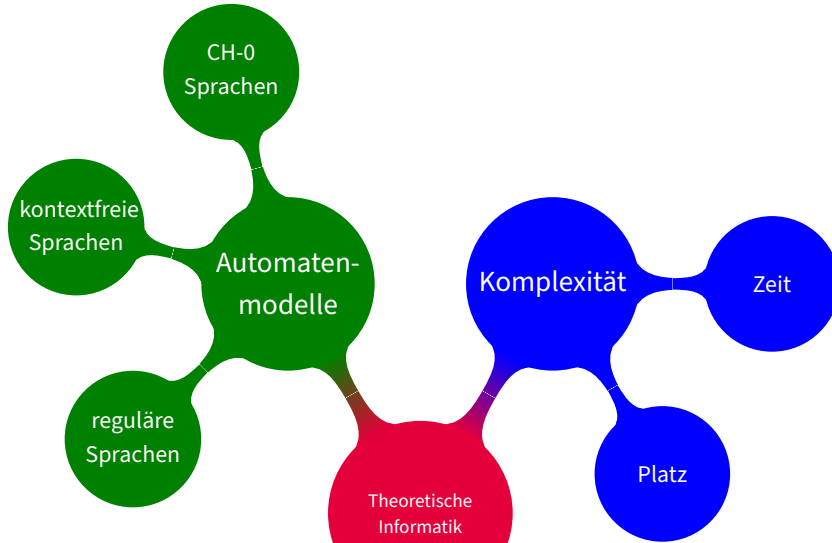


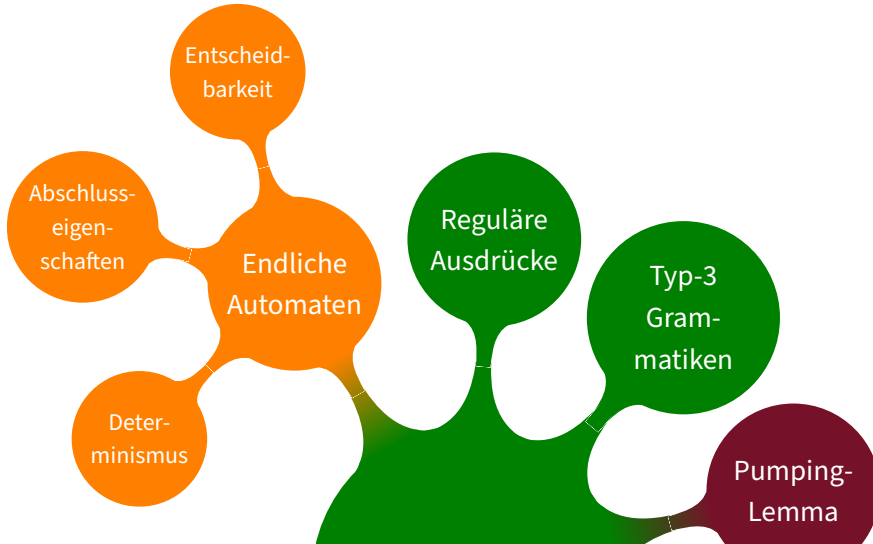
Wörter, Sprachen, Automaten

(Words, Languages, Automata)

Theoretische Informatik

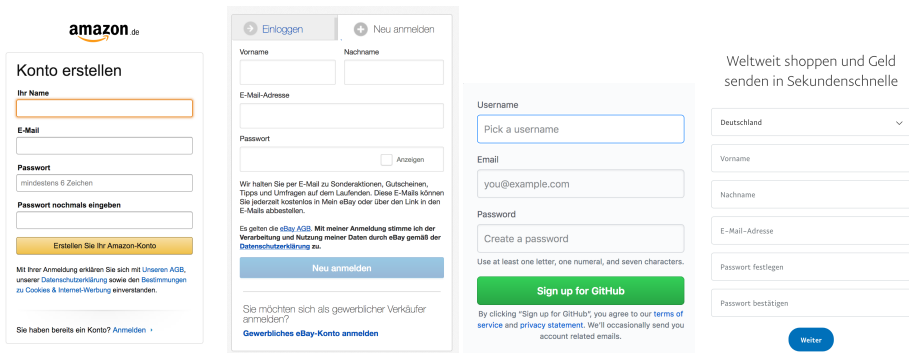
TH Lübeck





Alphabete, Wörter und Formale Sprachen

- Um das Lübecker Marzipan weltweit zu vermarkten, soll ein Web-Shop implementiert werden.
- Sie sind für die Prüfung der Kundeneingaben zuständig.



The image displays four screenshots of web registration forms, illustrating the user input fields and layout for a new web-shop:

- Amazon.de:** The form is titled "Konto erstellen". It includes fields for "Ihr Name", "E-Mail", "Passwort" (with a note "mindestens 6 Zeichen"), and "Passwort nochmals eingeben". A yellow button "Erstellen Sie Ihr Amazon-Konto" is at the bottom. Below the button, there is a disclaimer about terms and conditions.
- eBay:** The form has two tabs: "Einloggen" and "Neu anmelden". The "Neu anmelden" tab is active. It includes fields for "Vorname", "Nachname", "E-Mail-Adresse", and "Passwort" (with an "Anzeigen" checkbox). A blue button "Neu anmelden" is at the bottom. Below the button, there is a disclaimer about terms and conditions.
- GitHub:** The form includes fields for "Username" (with a placeholder "Pick a username"), "Email" (with a placeholder "you@example.com"), and "Password" (with a note "Use at least one letter, one numeral, and seven characters."). A green button "Sign up for GitHub" is at the bottom. Below the button, there is a disclaimer about terms and conditions.
- Worldwide Shopping:** A form titled "Weltweit shoppen und Geld senden in Sekundenschnelle". It includes a dropdown menu for "Deutschland", and fields for "Vorname", "Nachname", "E-Mail-Adresse", "Passwort festlegen", and "Passwort bestätigen". A blue button "Weiter" is at the bottom.

Anforderungen an Eingaben in Webshop?

Anforderungen an Eingaben in Webshop?

- Postleitzahlen 5 Ziffern in Deutschland
- Hausnummer Ziffern + 1 Buchstabe
- Kontonummer nur Ziffern
- keine gefährlichen Sonderzeichen (Anführungszeichen, JavaScript)
- ...

Für das Login soll der Nutzer sichere Passwörter wählen:

Password:

Strength:



Anforderungen an Passwörter:



Für das Login soll der Nutzer sichere Passwörter wählen:

Anforderungen an Passwörter:

- mindestens 6 Zeichen
- Großbuchstaben
- Kleinbuchstaben
- ...

Für das Login soll der Nutzer sichere Passwörter wählen:

Anforderungen an Passwörter:

- mindestens 6 Zeichen
- Großbuchstaben
- Kleinbuchstaben
- ...

Alphabet Zeichen, die Nutzer eintippen könnte

Wort Zeichenkette, die Nutzer als Passwort wählt

Sprache Menge der zulässigen Passwörter



Jetzt formal

Wie würden Sie Alphabet, Wort, Sprache definieren?



Beispiel (Lateinische Buchstaben)

A, B, C, D, E, F, G, H, I, J, K, L, M, N, O,
P, Q, R, S, T, U, V, W, X, Y, Z

Beispiel (Griechische Buchstaben)

$\alpha, \beta, \gamma, \delta, \varepsilon, \zeta, \eta, \theta, \iota, \kappa, \lambda, \mu, \nu, \xi, \omicron, \pi, \varrho,$
 $\sigma, \tau, \upsilon, \phi, \chi, \psi, \omega$

Beispiel (Lateinische Buchstaben)

A, B, C, D, E, F, G, H, I, J, K, L, M, N, O,
P, Q, R, S, T, U, V, W, X, Y, Z

Beispiel (Griechische Buchstaben)

$\alpha, \beta, \gamma, \delta, \varepsilon, \zeta, \eta, \theta, \iota, \kappa, \lambda, \mu, \nu, \xi, \omicron, \pi, \rho,$
 $\sigma, \tau, \upsilon, \phi, \chi, \psi, \omega$

Beispiel (Phoenizisches Alphabet)

𐤀, 𐤁, 𐤂, 𐤃, 𐤄, 𐤅, 𐤆, 𐤇, 𐤈, 𐤉, 𐤊, 𐤋, 𐤌, 𐤍,
𐤎, 𐤏, 𐤐, 𐤑, 𐤒, 𐤓, 𐤔, 𐤕, 𐤖, 𐤗, 𐤘

Beispiel (Hieroglyphen)



Beispiel (Zeichensätze)

ASCII, UTF-8, ISO8859-1, ...

Beispiel

...

```
<meta charset="utf-8"/>
```

...

Definition (Alphabet)

- Ein Alphabet Σ ist eine endliche und nicht-leere Menge. Die Elemente des Alphabets heißen Buchstaben.

Beispiel (Alphabet)

- $\Sigma_1 = \{a, b\}$
- $\Sigma_2 = \{a, \dots, z\}$ (Kleinbuchstaben)
- $\Sigma_3 = \{\mathbf{on}, \mathbf{off}\}$ enthält zwei Buchstaben **on** und **off** Ein Alphabet kann also eine beliebige endliche Menge sein.

Definition (Wort)

- Ein Wort $w = a_1 \dots a_n$ ist eine Sequenz von Buchstaben eines Alphabets.

Beispiel (Wort)

- $\Sigma_1 = \{a, b\}$: Wörter: *aaaa, ababab*
- $\Sigma_2 = \{a, .., z\}$ Wörter: *az, aazz, abc*
- $\Sigma_3 = \{\mathbf{on}, \mathbf{off}\}$ Wort: **onoffon**

Definition (Sprache)

- Eine Sprache L (Language) ist eine endliche oder unendliche Menge von Wörtern.

Beispiel (Sprache)

- $\Sigma_1 = \{a, b\}: L = \{a^n b^n \mid n \in \mathbb{N}\}$
- $\Sigma_2 = \{a, \dots, z\}: L = \{a^n \mid n \text{ ist durch } 3 \text{ teilbar}\}$
- $\Sigma_3 = \{\mathbf{on}, \mathbf{off}\}$ Die leere Sprache $\{\}$ oder die mit einem Wort $\{onon\}$.

```
String hello = "Hello";  
String world = "World";  
String greeting = hello + " " + world;  
String empty = "";  
...  
if (empty.isEmpty()){  
    ...  
}  
...  
for (int i = 0; i < greeting.length()){  
    ...  
}
```

```
String hello = "Hello";  
String world = "World";  
String greeting = hello + " " + world;  
String empty = "";  
...  
if (empty.isEmpty()){  
    ...  
}  
...  
for (int i = 0; i < greeting.length()){  
    ...  
}
```

- Strings in Java können leer sein.

```
String hello = "Hello";  
String world = "World";  
String greeting = hello + " " + world;  
String empty = "";  
...  
if (empty.isEmpty()){  
    ...  
}  
...  
for (int i = 0; i < greeting.length()){  
    ...  
}
```

- Strings in Java können leer sein.
- Strings kann man konkatenieren.

```
String hello = "Hello";  
String world = "World";  
String greeting = hello + " " + world;  
String empty = "";  
...  
if (empty.isEmpty()){  
    ...  
}  
...  
for (int i = 0; i < greeting.length()){  
    ...  
}
```

- Strings in Java können leer sein.
- Strings kann man konkatenieren.
- Die Länge eines Strings kann man ermitteln.

Operationen auf und Notationen zu Wörtern

- ε ist das leere Wort
- $w_1 \cdot w_2$ oder $w_1 w_2$ für das Hintereinanderschreiben von w_1 und w_2 (Konkatenation)
- w^n ist das Wort $\underbrace{w \dots w}_{n\text{-mal}}$ wobei $w^0 = \varepsilon$.
- $|w|$ ist die Länge des Wortes w
- Σ^* ist die Menge aller Wörter über dem Alphabet Σ (inkl. ε).

Operationen auf und Notationen zu Wörtern

- ε ist das leere Wort
- $w_1 \cdot w_2$ oder $w_1 w_2$ für das Hintereinanderschreiben von w_1 und w_2 (Konkatenation)
- w^n ist das Wort $\underbrace{w \dots w}_{n\text{-mal}}$ wobei $w^0 = \varepsilon$.
- $|w|$ ist die Länge des Wortes w
- Σ^* ist die Menge aller Wörter über dem Alphabet Σ (inkl. ε).

Beispiel (Konkatenation)

$w_1 = \text{Hello}, w_2 = \text{World}, w_3 = \varepsilon$

$w_1 w_3 w_2 = \text{HelloWorld}$

Operationen auf und Notationen zu Wörtern

- ε ist das leere Wort
- $w_1 \cdot w_2$ oder $w_1 w_2$ für das Hintereinanderschreiben von w_1 und w_2 (Konkatenation)
- w^n ist das Wort $\underbrace{w \dots w}_{n\text{-mal}}$ wobei $w^0 = \varepsilon$.
- $|w|$ ist die Länge des Wortes w
- Σ^* ist die Menge aller Wörter über dem Alphabet Σ (inkl. ε).

Beispiel (Konkatenation)

$w_1 = \text{Hello}, w_2 = \text{World}, w_3 = \varepsilon$
 $w_1 w_3 w_2 = \text{HelloWorld}$

Beispiel (Potenz)

$w = ab$
 $w^3 = ababab$

Operationen auf und Notationen zu Wörtern

- ε ist das leere Wort
- $w_1 \cdot w_2$ oder $w_1 w_2$ für das Hintereinanderschreiben von w_1 und w_2 (Konkatenation)
- w^n ist das Wort $\underbrace{w \dots w}_{n\text{-mal}}$ wobei $w^0 = \varepsilon$.
- $|w|$ ist die Länge des Wortes w
- Σ^* ist die Menge aller Wörter über dem Alphabet Σ (inkl. ε).

Beispiel (Konkatenation)

$w_1 = \text{Hello}, w_2 = \text{World}, w_3 = \varepsilon$
 $w_1 w_3 w_2 = \text{HelloWorld}$

Beispiel (Potenz)

$w = ab$
 $w^3 = ababab$

Beispiel (Länge)

$w = \text{hello}$
 $|w| = 5$

Konkatenation

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$

Alternativ: $L^{n+1} = L^n \circ L$

Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Konkatenation

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$

Alternativ: $L^{n+1} = L^n \circ L$

Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Beispiel (Konkatenation von Sprachen)

$$L_1 = \{ab, bb\}, L_2 = \{c, cc\},$$
$$L_1 \circ L_2 = \{abc,$$

Konkatenation

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$

Alternativ: $L^{n+1} = L^n \circ L$

Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Beispiel (Konkatenation von Sprachen)

$$L_1 = \{ab, bb\}, L_2 = \{c, cc\},$$
$$L_1 \circ L_2 = \{abc, abcc,$$

Konkatenation

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$

Alternativ: $L^{n+1} = L^n \circ L$

Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Beispiel (Konkatenation von Sprachen)

$$L_1 = \{ab, \textcolor{red}{bb}\}, L_2 = \{\textcolor{red}{c}, cc\},$$
$$L_1 \circ L_2 = \{abc, abcc, \textcolor{red}{bbc}, \textcolor{red}{bcc}\}$$

Konkatenation

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$

Alternativ: $L^{n+1} = L^n \circ L$

Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Beispiel (Konkatenation von Sprachen)

$$L_1 = \{ab, \textcolor{red}{bb}\}, L_2 = \{c, \textcolor{red}{cc}\},$$
$$L_1 \circ L_2 = \{abc, abcc, \textcolor{red}{bbc}, \textcolor{red}{bbcc}\}$$

Konkatenation

- $L_1 \circ L_2 = \{w_1w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$

Alternativ: $L^{n+1} = L^n \circ L$

Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Beispiel (Konkatenation von Sprachen)

$$L_1 = \{ab, bb\}, L_2 = \{c, cc\},$$
$$L_1 \circ L_2 = \{abc, abcc, bbc, bbcc\}$$

Beispiel (Potenz)

$$L = \{a, bb\}$$

$$L^3 = \{aaa, aabb, abba, \\ abbbb, bbaa, bbabb, \\ bbbba, bbbbbb\}$$

Konkatenation

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$

Alternativ: $L^{n+1} = L^n \circ L$

Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Beispiel (Konkatenation von Sprachen)

$$L_1 = \{ab, bb\}, L_2 = \{c, cc\},$$
$$L_1 \circ L_2 = \{abc, abcc, bbc, bbcc\}$$

Beispiel (Potenz)

$$L = \{a, bb\}$$

$$L^3 = \{aa\textcolor{red}{a}, aa\textcolor{red}{bb}, abba, \\ abbbb, bbaa, bbabb, \\ bbbba, bbbbbb\}$$

Konkatenation

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$

Alternativ: $L^{n+1} = L^n \circ L$

Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Beispiel (Konkatenation von Sprachen)

$$L_1 = \{ab, bb\}, L_2 = \{c, cc\},$$
$$L_1 \circ L_2 = \{abc, abcc, bbc, bbcc\}$$

Beispiel (Potenz)

$$L = \{a, bb\}$$

$$L^3 = \{a \textcolor{red}{a}a, aabb, abba, \\ \textcolor{red}{a}bbbb, bbaa, bbabb, \\ bbbba, bbbbbb\}$$

Konkatenation

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
(Konkatenation von Sprachen)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$
Alternativ: $L^{n+1} = L^n \circ L$
Menge aller Wörter, die man erhalten kann,
wenn man n Wörter aus L zusammensetzt.

Beispiel (Konkatenation von Sprachen)

$$L_1 = \{ab, bb\}, L_2 = \{c, cc\},$$
$$L_1 \circ L_2 = \{abc, abcc, bbc, bbcc\}$$

Beispiel (Potenz)

$$L = \{a, bb\}$$

$$L^3 = \{aaa, aabb, abba, \\ abbbb, bbaa, bbabb, \\ bbbba, bbbbbb\}$$

Stern

- $L^* = \bigcup_{i=0}^{\infty} L^i$

Menge aller Wörter, die man erhalten kann, wenn man beliebig viele (auch Null) Wörter aus L zusammensetzt. Das leere Wort ε ist immer in L^* enthalten.

Beispiel (Stern)

$$L = \{a, bb\}$$

$$L^* = \{\varepsilon, a, bb, \\ aa, abb, bbba, bbbb \\ aaa, aabb, \dots\}$$

Plus

- $L^+ = \bigcup_{i=1}^{\infty} L^i$

Menge aller Wörter, die man erhalten kann, wenn man beliebig viele – aber mindestens ein Wort – Wörter aus L zusammensetzt. Das leere Wort ε ist nur in L^+ , wenn es auch in L ist.

Beispiel (Plus)

$$L = \{a, bb\}$$

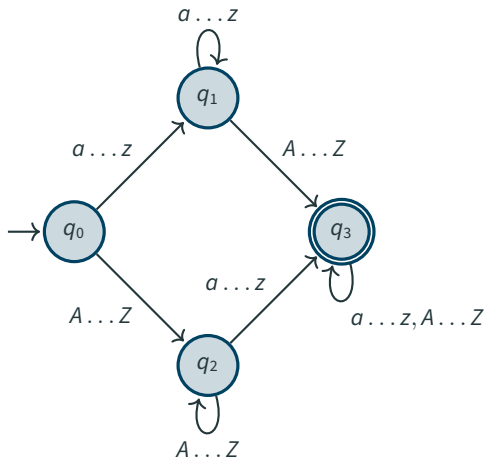
$$L^+ = \{a, bb, \\ aa, abb, bbaa, bbbb \\ aaa, aabb, \dots\}$$

- Zugelassen: Großbuchstaben, Kleinbuchstaben: $\Sigma = \{a, \dots, z, A, \dots, Z\}$
- Sprache:
 - $L_1 = \{w \mid w \text{ enthält Groß- und Kleinbuchstaben}\}$
 - $L_2 = \{w \mid |w| \geq 6\}$
 - $L_3 = L_1 \cap L_2 = \{w \mid w \text{ enthält Groß- und Kleinbuchstaben und Länge mind. 6}\}$

Deterministische endliche Automaten

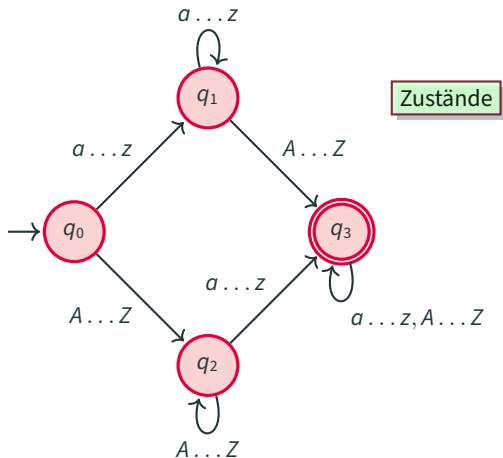
DEFINITION EINES ENDLICHEN AUTOMATEN I

Automat für die Sprache $L_1 = \{w \mid w \text{ enthält Groß- und Kleinbuchstaben}\}$



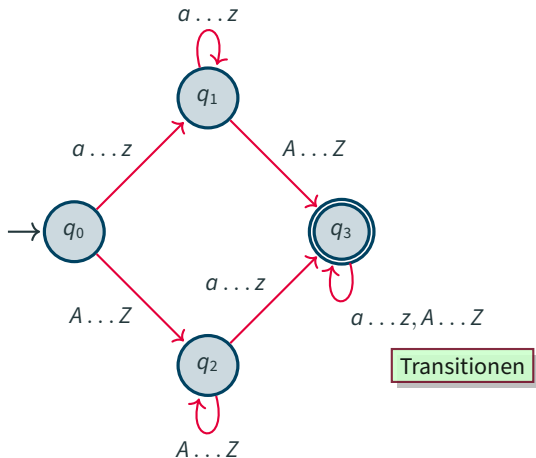
DEFINITION EINES ENDLICHEN AUTOMATEN I

Automat für die Sprache $L_1 = \{w \mid w \text{ enthält Groß- und Kleinbuchstaben}\}$



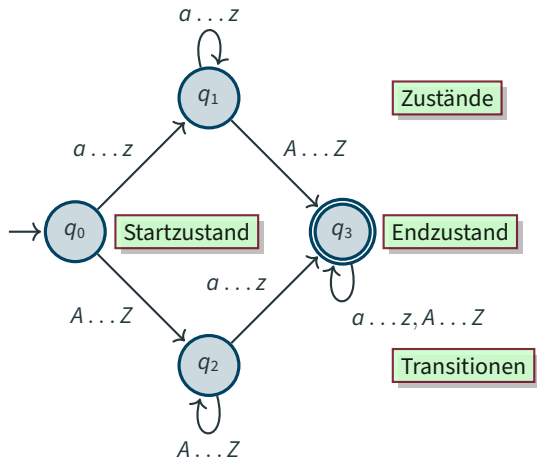
DEFINITION EINES ENDLICHEN AUTOMATEN I

Automat für die Sprache $L_1 = \{w \mid w \text{ enthält Groß- und Kleinbuchstaben}\}$



DEFINITION EINES ENDLICHEN AUTOMATEN I

Automat für die Sprache $L_1 = \{w \mid w \text{ enthält Groß- und Kleinbuchstaben}\}$



Definition (DEA)

Ein deterministischer endlicher Automat (engl. deterministic finite automaton), kurz DEA (engl. DFA), ist eine Struktur $A = (Q, \Sigma, \delta, q_0, F)$ mit folgenden Eigenschaften:

- Q ist eine endliche Menge von Zuständen (states)
- Σ ist eine endliche Menge, das Eingabealphabet (input alphabet)
- $\delta: Q \times \Sigma \rightarrow Q$ ist die Überföhrungsfunktion (transition function)
- $q_0 \in Q$ ist der Anfangszustand (start / initial state)
- $F \subseteq Q$ ist die Menge der Endzustände oder akzeptierenden Zustände (final / accepting states)

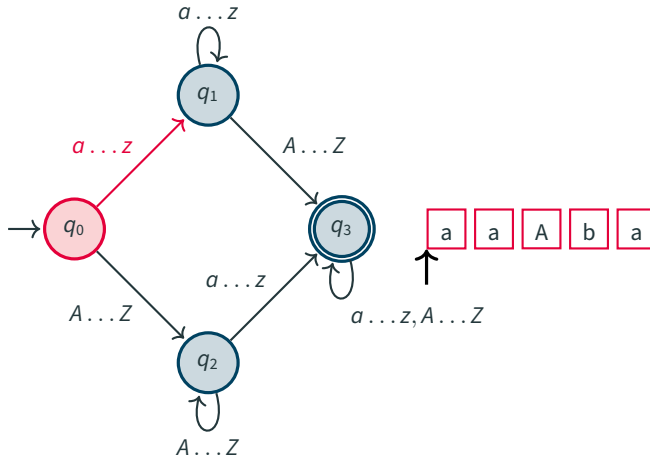
Notation

$q \xrightarrow{a} q'$ statt $\delta(q, a) = q'$

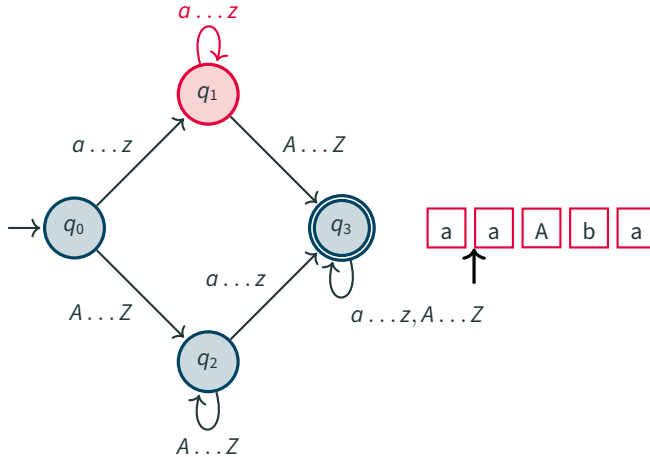
$$\delta : Q \times \Sigma \rightarrow Q$$

- Ausgehend von aktuellem Zustand ($Q \times \Sigma \rightarrow Q$)
- und Eingabebuchstabe ($Q \times \Sigma \rightarrow Q$)
- liefert δ *genau einen* (nicht null und nicht zwei) Nachfolgezustände ($Q \times \Sigma \rightarrow Q$)

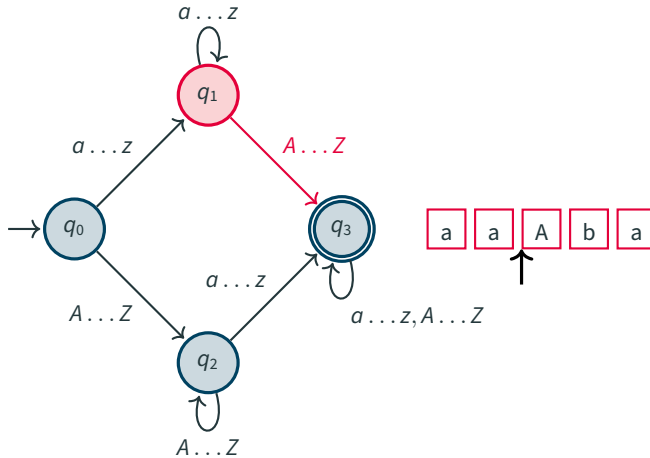
- Wie arbeitet ein Automat ein Wort $w = a_1 \dots a_n$ ab?



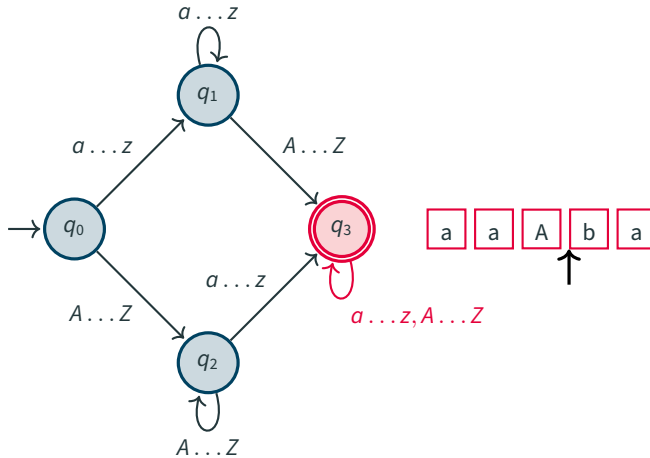
- Wie arbeitet ein Automat ein Wort $w = a_1 \dots a_n$ ab?



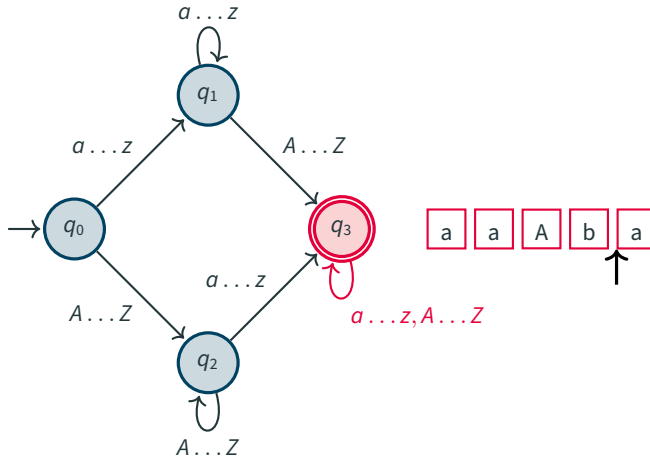
- Wie arbeitet ein Automat ein Wort $w = a_1 \dots a_n$ ab?



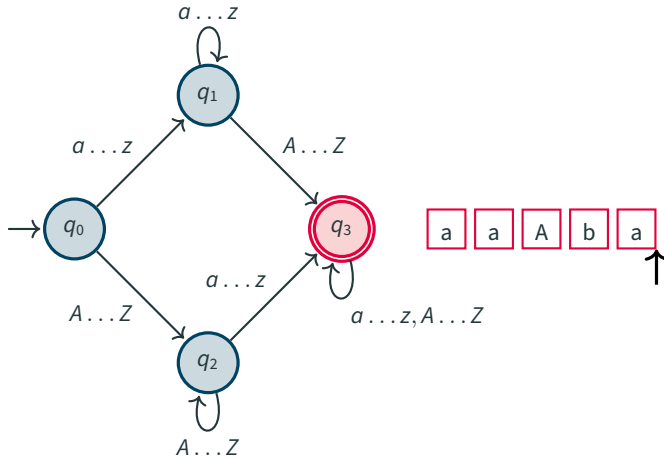
- Wie arbeitet ein Automat ein Wort $w = a_1 \dots a_n$ ab?



- Wie arbeitet ein Automat ein Wort $w = a_1 \dots a_n$ ab?



- Wie arbeitet ein Automat ein Wort $w = a_1 \dots a_n$ ab?



WIE ARBEITET EIN ENDLICHER AUTOMAT?

- Der Automat liest das Wort von links nach rechts und ändert entsprechend den Zustand.
- Für ein Eingabewort wird das Verhalten vollständig durch die besuchten Zustände beschrieben.

- Der Automat liest das Wort von links nach rechts und ändert entsprechend den Zustand.
- Für ein Eingabewort wird das Verhalten vollständig durch die besuchten Zustände beschrieben.

Definition (Lauf/Run)

Sei $A = (Q, \Sigma, \delta, q_0, F)$ ein DEA. Ein *Lauf* (Run) von A für ein Eingabewort $w = a_1 \dots a_n$ ist eine Folge $q_0, q_1, \dots, q_n$ von Zuständen, wobei

- Der Automat liest das Wort von links nach rechts und ändert entsprechend den Zustand.
- Für ein Eingabewort wird das Verhalten vollständig durch die besuchten Zustände beschrieben.
- Der Automat beginnt im Startzustand.
- Der Nachfolgezustand ist durch die Transitionsfunktion *eindeutig* festgelegt.

Definition (Lauf/Run)

Sei $A = (Q, \Sigma, \delta, q_0, F)$ ein DEA. Ein *Lauf* (Run) von A für ein Eingabewort $w = a_1 \dots a_n$ ist eine Folge $q_0, q_1, \dots, q_n$ von Zuständen, wobei

- q_0 der Startzustand von A ist und
- $\delta(q_{i-1}, a_i) = q_i$, d.h. A kommt von q_{i-1} mit Buchstabe a_i nach q_i .

Falls ein solcher Lauf existiert, schreiben wir $q_0 \xrightarrow{w} q_n$. Bei DEAs gibt es für jedes Wort genau einen Lauf. Ein Lauf heißt akzeptierend, falls er in einem akzeptierenden Zustand endet, d.h. $q_n \in F$.

- Der Automat liest das Wort von links nach rechts und ändert entsprechend den Zustand.
- Für ein Eingabewort wird das Verhalten vollständig durch die besuchten Zustände beschrieben.
- Der Automat beginnt im Startzustand.
- Der Nachfolgezustand ist durch die Transitionsfunktion *eindeutig* festgelegt.

Definition (Lauf/Run)

Sei $A = (Q, \Sigma, \delta, q_0, F)$ ein DEA. Ein *Lauf* (Run) von A für ein Eingabewort $w = a_1 \dots a_n$ ist eine Folge $q_0, q_1, \dots, q_n$ von Zuständen, wobei

- q_0 der Startzustand von A ist und
- $\delta(q_{i-1}, a_i) = q_i$, d.h. A kommt von q_{i-1} mit Buchstabe a_i nach q_i .

Falls ein solcher Lauf existiert, schreiben wir $q_0 \xrightarrow{w} q_n$. Bei DEAs gibt es für jedes Wort genau einen Lauf. **Ein Lauf heißt akzeptierend, falls er in einem akzeptierenden Zustand endet, d.h. $q_n \in F$.**

WELCHE SPRACHE AKZEPTIERT EIN DEA?

- Ein DEA akzeptiert genau die Wörter für die es einen akzeptierenden Lauf gibt.
- Das sind die Wörter, die in einen Endzustand führen.

- Ein DEA akzeptiert genau die Wörter für die es einen akzeptierenden Lauf gibt.
- Das sind die Wörter, die in einen Endzustand führen.

Definition (Akzeptierte Sprache)

Sei $A = (Q, \Sigma, \delta, q_0, F)$ ein DEA. Die von A akzeptierte Sprache ist

$$L(A) = \{w \in \Sigma^* \mid q_0 \xrightarrow{w} q_f \text{ und } q_f \in F\}$$

Konstruieren Sie deterministische endliche Automaten für folgende Sprachen:

Beispiel

Alphabet $\Sigma = \{a, b\}$:

- $L_1 = \{a^{3n} \in \Sigma^* \mid n \in \mathbb{N}\}$ (nur 'a', Anzahl durch 3 teilbar)
- $L_2 = \{(ab)^n \in \Sigma^* \mid n \in \mathbb{N}, n \geq 1\}$ ('ab' im Wechsel, mindestens einmal)
- $L_3 = \{bwab \in \Sigma^* \mid w \in \Sigma^*\}$ ('b' am Anfang, 'ab' am Ende)